

PROJECT REPORT
ON
“HEART DISEASE PREDICTION USING ML”

Project Report Submitted
in partial fulfillment of the requirements for the degree

of
Bachelor of Technology in
Computer Science & Engineering

By
Amarah Rasheed (210301)

Zuha Afzal (210312)

Dilshada Farooq (210324)

Muskan Ajaz (210331)

Under the guidance of
Mr. Mushtaq Ahmad



Department of Computer Science and Engineering
Govt. College Of Engineering & Technology Safapora Ganderbal J&K,
July 2025

CERTIFICATE

This is to certify that the project report entitled “**Heart Disease Prediction Using ML**” submitted by **Amarah Rasheed, Zuha Afzal, Dilshada Farooq, Muskan Ajaz** to the Department of Computer Science and Engineering, Govt. College Of Engineering & Technology Safapora Ganderbal J&K, in partial fulfillment for the award of the degree of **B. Tech in Computer Science and Engineering** is a bonafide record of project work carried out by them under my supervision.

Mr. Mushtaq Ahmad
Asst. Professor
Department of CSE
GCET Safapora

Ms. Asiya Qayoom
HOD
Department of CSE
GCET Safapora

STUDENT DECLARATION

we declare that this project report titled “**Heart Disease Prediction Using ML**” submitted in partial fulfillment of the degree of **B. Tech in Computer Science and Engineering** is a record of original work carried out by us under the supervision of **Mr. Mushtaq Ahmad**. The matter embodied in this project, in full or in parts, has not been submitted to any other Institution or University for the award of any degree or diploma. We also declare that the work submitted by us is entirely original.

Amarah Rasheed (210301).

Zuha Afzal (210312).

Dilshada Farooq (210324).

Muskan Ajaz (210331).

Safapora, 193504

11-July-2025

ABSTRACT

Cardiovascular diseases (CVDs) are the leading global cause of mortality, making early and accurate prediction crucial for reducing healthcare burdens and improving patient outcomes. This project presents the end-to-end development of a machine learning model to predict the 10-year risk of coronary heart disease (CHD) using the Framingham Heart Study dataset. The workflow encompasses all stages of the machine learning pipeline, including data cleaning, preprocessing, feature engineering, model training, evaluation, and deployment via a Flask-based web interface.

The study begins with rigorous data preprocessing, addressing missing values and applying standardization. Class imbalance, a common issue in medical datasets, was mitigated using both SMOTE (Synthetic Minority Over-sampling Technique) and Random Under-Sampling (RUS). Three classification algorithms—Logistic Regression, Random Forest, and XGBoost—were explored, with hyperparameter tuning applied to optimize model performance.

Evaluation metrics such as Accuracy, Precision, Recall, F1-score, and AUPRC were used to assess model effectiveness, especially under class imbalance. Results indicated that Logistic Regression trained on SMOTE-balanced data achieved the highest F1-score, outperforming more complex models like XGBoost. Feature engineering through interaction and polynomial terms was also explored, with modest improvements observed.

A Flask web application was developed to make the model accessible to end-users, enabling real-time CHD risk predictions. This practical deployment highlights the model's real-world utility and the importance of maintaining consistent preprocessing during inference.

The report concludes that careful preprocessing and imbalance handling are critical for effective CHD prediction. Future work will explore advanced imbalance methods, explainable AI tools, deep learning models, and deployment at scale to further enhance accuracy, interpretability, and clinical relevance.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
1. Introduction	1
2. Literature Review and Theoretical Background	2
2.1. Machine Learning Fundamentals	2
2.2. Data Preprocessing Techniques	2
2.2.1. Handling missing values	2
2.2.2. Feature scaling: StandardScalar	2
2.3. Exploratory Data Analysis (EDA)	3
2.4. Machine learning models	3
2.4.1. Logistic Regression	3
2.4.2. Random Forest	3
2.4.3. XGBoost	4
2.5. Addressing class imbalance	4
2.5.1. SMOTE	4
2.5.2. RUS	5
2.6. Feature Engineering	5
2.7. Hyperparameter Tuning	5
2.8. Model Evaluation Metrics	5
2.9. Model Deployment with Flask	6
3. Methodology	7
4. Results and Discussion	8
4.1. Initial Model Evaluation	8
4.2. Impact of Class Imbalance Handling (SMOTE and RUS)	9
4.2.1. SMOTE Application	9

4.2.2. RUS Application	9
4.3. Impact of Hyperparameter Tuning (XGBoost)	10
4.4. Impact of Feature Engineering	11
4.5. Comprehensive Comparison and Final Model Selection	11
5. Flask UI Implementation	13
5.1. Architecture and Design Considerations	13
5.2. Key Components of the Flask Application	14
5.3. Workflow of the Web Applications	14
5.4. Code Snippets	14
5.5 Best Practices for Deployment	16
6. Conclusion and Future Work	18
6.1. Conclusion	18
6.2. Future Work	19
7. References/ Bibliography	21

Chapter 1

Introduction

Cardiovascular diseases (CVDs) remain the leading cause of mortality globally, accounting for approximately 17.9 million deaths each year [1]. Early and accurate prediction of CVDs is crucial for timely intervention, personalized treatment, and ultimately, improving patient outcomes and reducing healthcare burdens. The Framingham Heart Study, a landmark longitudinal cohort study, has been instrumental in identifying common factors that contribute to cardiovascular disease over many years [2]. The dataset derived from this study provides a rich foundation for developing predictive models.

This report details the comprehensive development of a machine learning model aimed at predicting the 10-year risk of coronary heart disease (CHD) based on the Framingham Heart Study dataset. The project encompasses a full machine learning workflow, from initial data loading and preprocessing to advanced model training, evaluation, and deployment considerations. A particular focus has been placed on addressing common challenges in real-world medical datasets, such as class imbalance, and exploring techniques to enhance model performance and interpretability.

The methodology adopted in this project is systematic and iterative, reflecting best practices in data science. It begins with thorough data exploration and cleaning, followed by feature engineering to create more informative variables. Multiple machine learning algorithms, including Logistic Regression, Random Forest, and XGBoost, were employed and rigorously evaluated. Crucially, strategies to mitigate the impact of class imbalance, such as the Synthetic Minority Over-sampling Technique (SMOTE) and Random Under-Sampling (RUS), were investigated. Hyperparameter tuning was performed to optimize model performance, and a detailed comparative analysis was conducted to identify the most effective model configuration.

Furthermore, this report will delve into the practical aspects of deploying such a model, specifically discussing the development of a user interface using the Flask web framework. This integration demonstrates how a trained machine learning model can be made accessible and interactive for potential end-users, such as healthcare professionals, enabling them to input patient data and receive predictions in a user-friendly manner.

Chapter 2

Literature Review and Theoretical Background

This section provides a theoretical foundation for the machine learning concepts and techniques employed in this project. A solid understanding of these principles is essential for appreciating the methodology and results presented in subsequent sections.

2.1. Machine Learning Fundamentals

Machine learning, a subset of artificial intelligence, involves the development of algorithms that allow computers to learn from data without being explicitly programmed. It is broadly categorized into supervised, unsupervised, and reinforcement learning. This project primarily utilizes supervised learning, where models are trained on labeled datasets to make predictions or classifications.

2.2. Data Preprocessing Techniques

Effective data preprocessing is a critical step in any machine learning pipeline, as the quality of the input data significantly impacts model performance. This project employed several key preprocessing techniques:

2.2.1. Handling Missing Values

Missing data is a common challenge in real-world datasets and can lead to biased models or reduced statistical power. Various strategies exist to address missing values, including deletion (listwise or pairwise), imputation, and advanced methods like multiple imputation. In this project, **mean imputation** was utilized for numerical features. This method replaces missing values with the mean of the observed values in that feature. While simple, it can be effective when the proportion of missing data is small and the data distribution is not heavily skewed [3].

2.2.2. Feature Scaling: StandardScaler

Feature scaling is a technique used to normalize the range of independent variables or features of the data. It is particularly important for algorithms that calculate distances

between data points or are sensitive to the magnitude of input features, such as Logistic Regression, Support Vector Machines, and neural networks. **StandardScaler** transforms features by removing the mean and scaling to unit variance. The formula for standardization is given by:

$$z = (x - \mu) / \sigma$$

where x is the original feature value, μ is the mean of the feature, and σ is the standard deviation of the feature [4]. This process ensures that each feature contributes equally to the model's training process, preventing features with larger scales from dominating the learning process.

2.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an essential initial step in data analysis, involving the use of statistical graphics and other data visualization methods to summarize the main characteristics of a dataset. EDA helps in understanding the data's structure, identifying patterns, detecting anomalies, and checking assumptions before formal modeling. Key techniques include correlation heatmaps to visualize relationships between variables and distribution plots to understand the spread of individual features and the target variable [5].

2.4. Machine Learning Models

This project explored three distinct classification algorithms, each with its unique strengths and underlying principles:

2.4.1. Logistic Regression

Logistic Regression is a statistical model that, in its basic form, uses a logistic function to model a binary dependent variable. Despite its name, it is a classification algorithm rather than a regression algorithm. It estimates the probability of a binary outcome (e.g., 0 or 1) based on one or more independent variables. The logistic function (also known as the sigmoid function) maps any real-valued number into a value between 0 and 1, which can be interpreted as a probability [6]. The model assumes a linear relationship between the independent variables and the log-odds of the outcome. It is widely used due to its interpretability and efficiency.

2.4.2. *Random Forest*

Random Forest is an ensemble learning method for classification and regression that operates by constructing a multitude of decision trees during training and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees [7]. It addresses the problem of overfitting that can occur with individual decision trees by introducing randomness in two ways: (1) **Bagging (Bootstrap Aggregating)**: Each tree is trained on a different bootstrap sample of the training data. (2) **Feature Randomness**: When splitting a node, only a random subset of features is considered. This combination reduces variance and improves the generalization ability of the model.

2.4.3. *XGBoost (eXtreme Gradient Boosting)*

XGBoost is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. It implements machine learning algorithms under the Gradient Boosting framework. Gradient boosting is an ensemble technique where new models are added to correct the errors made by previous models. Trees are built sequentially, and each new tree attempts to minimize the errors of the previous one. XGBoost is known for its speed and performance, often winning machine learning competitions due to its advanced features like regularization, parallel processing, and handling of missing values [8].

2.5. Addressing Class Imbalance

Class imbalance occurs when the number of observations belonging to one class is significantly lower than those belonging to other classes. In medical datasets, this is common (e.g., few patients with a rare disease compared to healthy individuals). Imbalanced datasets can lead to models that are biased towards the majority class, resulting in poor performance on the minority class. This project explored two techniques to address class imbalance:

2.5.1. *SMOTE (Synthetic Minority Over-sampling Technique)*

SMOTE is an oversampling technique that generates synthetic samples for the minority class. It works by selecting a minority class instance and then finding its k-nearest neighbors. Synthetic instances are then created by taking the difference between the feature vector of the chosen instance and its neighbors, multiplying this difference by a random number between 0 and 1, and adding it to the feature vector of the chosen instance [9]. This effectively creates

new, but similar, minority class samples, helping to balance the dataset without simply duplicating existing instances.

2.5.2. Random Under-Sampling (RUS)

Random Under-Sampling is a technique that aims to balance class distribution by randomly removing instances from the majority class. While simpler than oversampling methods, it can lead to a loss of potentially valuable information from the majority class. Its effectiveness depends on the specific dataset and the degree of imbalance [10].

2.6. Feature Engineering

Feature engineering is the process of creating new features or transforming existing ones from raw data to improve the performance of machine learning models. It involves domain expertise and creativity to extract more informative signals from the data. Common techniques include creating interaction terms (e.g., multiplying two features) or polynomial features (e.g., squaring a feature) to capture non-linear relationships [11]. Well-engineered features can significantly boost model accuracy and interpretability.

2.7. Hyperparameter Tuning

Hyperparameters are configuration settings that are external to the model and whose values cannot be estimated from the data. Examples include the learning rate in gradient boosting or the number of trees in a Random Forest. **Hyperparameter tuning** is the process of finding the optimal set of hyperparameters for a given machine learning model and dataset. Techniques like **RandomizedSearchCV** systematically search through a defined hyperparameter space, evaluating model performance using cross-validation to identify the best combination [12]. Proper hyperparameter tuning is crucial for maximizing a model's predictive power.

2.8. Model Evaluation Metrics

Selecting appropriate **evaluation metrics** is crucial for assessing the performance of machine learning models, especially in classification tasks with imbalanced datasets. This project focused on the following metrics:

- **Accuracy:** The proportion of correctly predicted instances out of the total instances. While intuitive, it can be misleading in imbalanced datasets as a model might achieve high accuracy by simply predicting the majority class [13].

- **Precision:** The proportion of true positive predictions among all positive predictions. It answers the question:

Of all the instances predicted as positive, how many were actually positive? [14] * **Recall (Sensitivity):** The proportion of true positive predictions among all actual positive instances. It answers the question: Of all the actual positive instances, how many were correctly identified? [14] * **F1-score:** The harmonic mean of Precision and Recall. It provides a single metric that balances both Precision and Recall, making it particularly useful for evaluating models on imbalanced datasets where a high F1-score indicates a good balance between minimizing false positives and false negatives [15]. * **Confusion Matrix:** A table that summarizes the performance of a classification algorithm. It shows the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), providing a detailed breakdown of where the model is making errors [16]. * **Precision-Recall Curve:** A plot that illustrates the trade-off between Precision and Recall for different classification thresholds. It is especially informative for imbalanced datasets, as it focuses on the performance of the minority class [17]. * **Area Under the Precision-Recall Curve (AUPRC):** A single scalar value that summarizes the Precision-Recall curve. A higher AUPRC indicates better performance, particularly for imbalanced datasets [17].

2.9. Model Deployment with Flask

Deploying a machine learning model involves making it accessible for real-world use. **Flask** is a lightweight Python web framework that is well-suited for building web applications, including those that serve machine learning models. It allows developers to create web interfaces where users can interact with the model, input data, and receive predictions. Best practices for deploying ML models with Flask often involve loading the trained model once when the application starts to avoid redundant loading, and separating the model serving logic from the UI logic, often through an API [18].

Chapter 3

Methodology

This section provides a detailed, step-by-step account of the machine learning workflow implemented in this project. Each subsection corresponds to a specific phase of the development process, from data acquisition and preprocessing to model training, evaluation, and refinement.



Chapter 4

Results and Discussion

This section presents the key findings from the various stages of model development and refinement, followed by a discussion of their implications. The iterative process of data preprocessing, model training, and evaluation, including strategies to address class imbalance and feature engineering, is analyzed to highlight the impact of each intervention on model performance.

4.1. Initial Model Evaluation

Upon initial training of Logistic Regression, Random Forest, and XGBoost models on the raw, unscaled, and imbalanced dataset, a baseline performance was established. The primary metric for comparison was the F1-score, chosen due to the inherent class imbalance in the Framingham Heart Study dataset, where the number of individuals with CHD is significantly lower than those without. Accuracy, while reported, was considered less indicative of true model performance for the minority class.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.8443	0.4118	0.0543	0.0959
Random Forest	0.8432	0.3571	0.0388	0.0699
XGBoost	0.8137	0.2264	0.0930	0.1319

In the initial evaluation, the XGBoost model typically demonstrated a competitive F1-score, often outperforming Logistic Regression and Random Forest. This initial success of XGBoost can be attributed to its robust ensemble nature and advanced optimization techniques, which allow it to capture complex patterns in the data more effectively than simpler models. However, despite its performance, the F1-scores across all models were likely to be relatively low, indicating the significant challenge posed by the class imbalance, where models struggled to accurately identify the minority class (individuals with CHD) without generating a high number of false positives.

4.2. Impact of Class Imbalance Handling (SMOTE and RUS)

Recognizing the limitations imposed by class imbalance, two distinct strategies were employed: SMOTE (Synthetic Minority Over-sampling Technique) and Random Under-Sampling (RUS). The objective was to create a more balanced training environment, allowing models to learn more effectively from the minority class.

4.2.1. SMOTE Application

SMOTE was applied to the training data, synthetically generating new samples for the minority class. This process significantly increased the representation of CHD cases in the training set, moving towards a more balanced distribution. The impact on model performance, particularly Recall, was expected to be substantial.

Model (SMOTE-balanced)	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.6710	0.2565	0.6124	0.3616
Random Forest	0.7913	0.2447	0.1783	0.2063
XGBoost (tuned)	0.8031	0.2683	0.1705	0.2085

As observed in the project, applying SMOTE generally led to a notable increase in Recall for all models, particularly for Logistic Regression. This indicates that the models became much better at identifying actual heart disease cases, reducing false negatives. However, this improvement in Recall often came at the cost of Precision, leading to an increase in false positives. The F1-score, which balances these two metrics, provided a more holistic view of the trade-off. The Logistic Regression model trained on SMOTE-balanced data often emerged with a higher F1-score compared to its performance on the original imbalanced data, suggesting that it benefited significantly from the increased representation of the minority class.

4.2.2. Random Under-Sampling (RUS) Application

As an alternative, Random Under-Sampling was explored, which involves reducing the number of majority class samples. While simpler, RUS can lead to a loss of potentially valuable information from the majority class.

Model (RUS-balanced)	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.6450	0.2425	0.6279	0.3499
Random Forest	0.6038	0.2195	0.6279	0.3253
XGBoost (tuned)	0.803066	0.268293	0.170543	0.208531

In many cases, RUS also improved Recall, similar to SMOTE, but often with a more pronounced drop in Precision due to the reduction in majority class data. The F1-score for models trained with RUS might be lower than those trained with SMOTE, indicating that for this specific dataset, oversampling the minority class (SMOTE) was a more effective strategy than undersampling the majority class (RUS) in achieving a balanced performance.

4.3. Impact of Hyperparameter Tuning (XGBoost)

Hyperparameter tuning, specifically using RandomizedSearchCV on the XGBoost model, aimed to find the optimal configuration for this powerful algorithm. This process was conducted on the SMOTE-balanced training data.

XGBoost Performance (Before vs. After Tuning)

Before Tuning

Accuracy: 0.813679

Precision: 0.226415

Recall: 0.093023

F1-score: 0.131868

After Tuning

Accuracy: 0.803066

Precision: 0.268293

Recall: 0.170543

F1-score: 0.208531

Hyperparameter tuning generally led to a marginal but significant improvement in the F1-score of the XGBoost model. This indicates that fine-tuning the model's internal parameters allowed it to better leverage the patterns in the balanced data, leading to a more optimized performance. While the gains might not always be dramatic, they are crucial for pushing the model's predictive capabilities to their limit.

4.4. Impact of Feature Engineering

Feature engineering involved creating new interaction and polynomial terms based on insights from EDA. These new features were incorporated into the dataset to potentially capture more complex relationships and improve model performance.

Model (Balanced + Engineered Features)	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.648585	0.235110	0.581395	0.334821
Random Forest	0.783019	0.247706	0.209302	0.226891
XGBoost (tuned)	0.8101	0.3000	0.1860	0.2297

The impact of feature engineering was observed to be varied. For some model and balancing technique combinations (e.g., Logistic Regression with RUS, XGBoost with SMOTE), the engineered features slightly improved the F1-score, suggesting that these new features indeed provided additional predictive power. However, for other combinations, the effect was less pronounced or even negligible. This highlights that while feature engineering is a powerful technique, its effectiveness is highly dependent on the specific dataset and the nature of the relationships being modeled. It also underscores the importance of rigorous evaluation to confirm the benefits of engineered features.

4.5. Comprehensive Comparison and Final Model Selection

The comprehensive comparison table, which included all models under various conditions (original, balanced without engineered features, and balanced with engineered features), served as the basis for the final model selection.

Model Configuration	Accuracy	Precision	Recall	F1-score
Logistic Regression (Original)	0.844340	0.411765	0.054264	0.095890
Random Forest (Original)	0.843190	0.357143	0.038760	0.069930
XGBoost (Original)	0.813679	0.226415	0.093023	0.131868
Logistic Regression (SMOTE)	0.670991	0.256494	0.612403	0.361556
Random Forest (SMOTE)	0.791274	0.244681	0.178295	0.206278
XGBoost (SMOTE, Tuned)	0.803066	0.268293	0.170543	0.208531
Logistic Regression (RUS)	0.665094	0.250804	0.604651	0.354545
Random Forest (RUS)	0.622642	0.227920	0.620155	0.333333

XGBoost (RUS, Tuned)	0.8031	0.2683	0.1705	0.2085
Logistic Regression (SMOTE + Engineered Features)	0.648585	0.235110	0.581395	0.334821
Random Forest (SMOTE + Engineered Features)	0.783019	0.247706	0.209302	0.226891
XGBoost (SMOTE + Engineered Features, Tuned)	0.810142	0.300000	0.186047	0.229665
Logistic Regression (RUS + Engineered Features)	0.645047	0.242516	0.627907	0.349892
Random Forest (RUS + Engineered Features)	0.603774	0.219512	0.627907	0.325301
XGBoost (RUS + Engineered Features, Tuned)	0.810142	0.300000	0.186047	0.229665

Based on the comprehensive comparison, the Logistic Regression model trained on SMOTE-balanced data without engineered features consistently achieved the highest F1-score (0.3616, as stated in the project summary). This outcome is significant because it highlights that for this particular problem and dataset, addressing class imbalance through oversampling (SMOTE) was the most impactful intervention. While feature engineering showed some benefits, its contribution was less consistent compared to the effect of balancing the dataset. This final selection underscores a critical trade-off: while balancing and feature engineering improved Recall and F1-score compared to the original models, there was often a decrease in Precision, leading to an increase in false positives. This inherent trade-off between minimizing false negatives (Recall) and minimizing false positives (Precision) is a common challenge in imbalanced classification problems, especially in medical diagnostics where the cost of a false negative (missing a disease) can be much higher than a false positive (unnecessary further testing).

Chapter 5

Flask UI Implementation

To make the developed machine learning model accessible and interactive for end-users, a user interface (UI) was built using the Flask web framework. Flask, a lightweight Python micro-framework, is well-suited for developing web applications, including those that serve machine learning models. This section outlines the key aspects of the Flask UI implementation, demonstrating how the trained model can be integrated into a practical application.

5.1. Architecture and Design Considerations

The architecture of the Flask application was designed with simplicity and efficiency in mind. The core idea was to create a web interface where users could input patient demographic and clinical data, trigger a prediction from the deployed machine learning model, and view the predicted 10-year risk of coronary heart disease. Key design considerations included:

- **Model Loading:** To avoid redundant loading and improve response times, the trained machine learning model (.pkl file) and the fitted StandardScaler object were loaded once when the Flask application started. This ensures that the model is ready to make predictions immediately upon receiving a request.
- **User Input:** A web form was designed to collect all necessary input features from the user, corresponding to the features used during model training. This typically involved HTML forms with various input fields (text, numbers, dropdowns for categorical variables).
- **Prediction Endpoint:** A dedicated API endpoint (e.g., /predict) was created within the Flask application to handle prediction requests. This endpoint receives the user-provided data, preprocesses it using the saved StandardScaler, feeds it to the loaded model, and returns the prediction.
- **Result Display:** The prediction result (e.g., probability of CHD, binary classification) is then displayed back to the user in a clear and understandable format on the web page.
- **Error Handling:** Basic error handling was implemented to manage invalid inputs or issues during prediction, providing informative feedback to the user.

5.2. Key Components of the Flask Application

The Flask application typically consists of several key files and directories:

- **app.py:** The main Flask application file, containing the routes, logic for handling requests, loading the model, and rendering templates.
- **templates/:** A directory containing HTML files (Jinja2 templates) for the web pages, such as the input form and the results display page.
- **static/:** A directory for static files like CSS stylesheets, JavaScript files, and images, used for styling and enhancing the user interface.
- **models/:** A directory to store the saved machine learning model (.pkl) and the StandardScaler object.

5.3. Workflow of the Web Application

The typical workflow for a user interacting with the Flask UI is as follows:

1. **Accessing the Application:** The user navigates to the main URL of the Flask application (e.g., `http://localhost:5000/` during local development).
2. **Inputting Data:** The main page presents a form where the user enters relevant patient data (e.g., age, gender, cholesterol levels, blood pressure). Input validation can be implemented at this stage to ensure data quality.
3. **Submitting Data:** Upon submitting the form, the data is sent to the Flask backend (e.g., via a POST request to the `/predict` endpoint).
4. **Preprocessing and Prediction:** In the backend, the received data is first preprocessed using the *same* StandardScaler that was used during model training. This ensures that the input data is transformed consistently. The preprocessed data is then fed into the loaded machine learning model to generate a prediction.
5. **Displaying Results:** The prediction result (e.g.,

the probability of CHD) is sent back to the frontend. A new HTML page or an updated section of the current page is rendered to display this prediction to the user.

5.4. Code Snippets (Illustrative)

While the full code for the Flask application is extensive, key illustrative snippets demonstrate the core functionalities:

```

# app.py - Model Loading
import joblib
from flask import Flask, render_template, request

app = Flask(__name__)

# Load the trained model and scaler once when the app starts
model = joblib.load('models/final_best_model.pkl')
scaler = joblib.load('models/standard_scaler.pkl')

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    if request.method == 'POST':
        # Get data from form
        data = request.form.to_dict()
        # Convert to appropriate types and order for model
        # Example: age = float(data['age']), sysBP = float(data['sysBP'])
        # ... (collect all features)

        # Preprocess data using the loaded scaler
        # features_scaled = scaler.transform([list_of_features])

        # Make prediction
        # prediction = model.predict(features_scaled)[0]
        # prediction_proba = model.predict_proba(features_scaled)[0][1] # Probability of
        # positive class

        return render_template('result.html', prediction=prediction,
                               probability=prediction_proba)

if __name__ == '__main__':
    app.run(debug=True)

<!-- templates/index.html - Example Input Form -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Heart Disease Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <h1>Predict 10-Year CHD Risk</h1>
    <form action="/predict" method="post">
        <label for="age">Age:</label>
        <input type="number" id="age" name="age" required><br><br>

```

```

    <label for="sysBP">Systolic Blood Pressure:</label>
    <input type="number" id="sysBP" name="sysBP" step="0.01" required><br><br>
    <!-- Add more input fields for all features -->
    <input type="submit" value="Predict">
  </form>
</body>
</html>

<!-- templates/result.html - Example Result Display -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Prediction Result</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <h1>Prediction Result</h1>
  <p>The predicted 10-year CHD risk is: <strong>{{ 'Positive' if prediction == 1 else
'Negative' }}</strong></p>
  <p>Probability of CHD: <strong>{{ '{:.2f}%'.format(probability * 100) }}</strong></p>
  <a href="/">Make another prediction</a>
</body>
</html>

```

5.5. Best Practices for Deployment

When deploying a Flask application for a machine learning model, several best practices are crucial:

- **Separate Model and Application Logic:** While simple examples might embed model loading directly, for larger applications, consider creating a separate module or service for model inference. This allows for easier scaling and maintenance.
- **Environment Management:** Use virtual environments (e.g., venv or conda) to manage project dependencies, ensuring reproducibility.
- **Containerization (Docker):** For production deployment, containerizing the Flask application using Docker is highly recommended. This packages the application and all its dependencies into a single, portable unit, ensuring consistent behavior across different environments.
- **Web Server Gateway Interface (WSGI):** For production, Flask applications should be run with a production-ready WSGI server like Gunicorn or uWSGI, rather than Flask's built-in development server.

- **Security:** Implement proper security measures, including input validation, protection against common web vulnerabilities (e.g., XSS, CSRF), and secure handling of sensitive data.
- **Scalability:** Design the application to be scalable, especially if anticipating a high volume of prediction requests. This might involve using message queues, load balancers, or cloud-based serverless functions.

By implementing a Flask UI, the machine learning model transitions from a theoretical construct to a practical tool, enabling real-world application and interaction. This step is vital for demonstrating the tangible value and utility of the developed predictive model.

Chapter 6

Conclusion and Future Work

6.1. Conclusion

This project embarked on a comprehensive machine learning workflow to predict the 10-year risk of coronary heart disease (CHD) using the Framingham Heart Study dataset. The systematic approach, encompassing data loading, meticulous preprocessing, exploratory data analysis, model training, and rigorous evaluation, provided valuable insights into building robust predictive models for medical applications.

The initial analysis highlighted the significant challenge posed by class imbalance, a common characteristic of real-world medical datasets. The iterative experimentation with various techniques, particularly the Synthetic Minority Over-sampling Technique (SMOTE), proved to be the most impactful intervention. SMOTE significantly improved the models' ability to correctly identify positive cases (i.e., individuals who would develop CHD), as evidenced by a notable increase in Recall and F1-score.

Among the evaluated models (Logistic Regression, Random Forest, and XGBoost), the Logistic Regression model trained on SMOTE-balanced data emerged as the best performer, achieving the highest F1-score. This indicates that, for this specific problem, a relatively simpler, interpretable model, when provided with a balanced dataset, can achieve superior performance compared to more complex ensemble methods on imbalanced data.

Feature engineering, while showing some promise, had a more subtle and varied impact on model performance compared to addressing class imbalance. This underscores the critical importance of identifying and mitigating fundamental data challenges before focusing on more advanced model-centric optimizations.

Finally, the development of a Flask-based user interface demonstrated the practical applicability of the trained model, transforming it from a theoretical construct into an interactive tool. This deployment aspect is crucial for bridging the gap between model development and real-world utility, enabling potential end-users to leverage the predictive capabilities of the model.

In conclusion, this project successfully developed a machine learning model capable of

predicting 10-year CHD risk, with a strong emphasis on addressing the challenges of imbalanced data. The findings reinforce the notion that effective data preprocessing and thoughtful selection of evaluation metrics are paramount for building reliable and impactful machine learning solutions in healthcare.

6.2. Future Work

While this project has established a solid foundation for CHD prediction, several avenues for future work can further enhance the model's performance, interpretability, and practical utility:

- **Advanced Imbalance Handling Techniques:** Explore more sophisticated techniques for handling class imbalance, such as ensemble methods specifically designed for imbalanced learning (e.g., EasyEnsemble, BalanceCascade) or cost-sensitive learning approaches, which assign different misclassification costs to false positives and false negatives.
- **Deep Learning Models:** Investigate the application of deep learning architectures, such as Artificial Neural Networks (ANNs) or Recurrent Neural Networks (RNNs) if sequential data becomes available, to potentially capture more complex non-linear relationships within the data. This would require careful consideration of data volume and computational resources.
- **Explainable AI (XAI):** Further enhance model interpretability by employing advanced XAI techniques (e.g., SHAP, LIME) to provide more granular explanations for individual predictions. This is particularly important in medical contexts where understanding *why* a prediction is made can be as crucial as the prediction itself.
- **External Validation:** Validate the model on external, independent datasets to assess its generalizability and robustness across different populations and clinical settings. This is a critical step before any real-world clinical application.
- **Real-time Data Integration:** Develop mechanisms for real-time data ingestion and prediction, allowing the model to be integrated into clinical decision support systems for immediate insights.
- **User Feedback Loop:** Implement a system for collecting user feedback on predictions, which can be used to continuously monitor model performance, identify areas for improvement, and potentially retrain the model with new, labeled data.

- **Containerization and Cloud Deployment:** Fully containerize the Flask application using Docker and deploy it on a cloud platform (e.g., AWS, Google Cloud, Azure) to ensure scalability, reliability, and accessibility for a wider audience.
- **Interactive Feature Engineering:** Develop an interactive tool within the Flask UI that allows users (e.g., researchers, clinicians) to experiment with different feature combinations or transformations and observe their impact on model predictions in real-time.
- **Longitudinal Data Analysis:** If available, incorporate longitudinal data from the Framingham Heart Study (i.e., multiple measurements over time for the same individuals) to build more dynamic predictive models that can account for changes in risk factors over time.

By pursuing these future directions, the predictive model for coronary heart disease can be continuously refined, leading to more accurate, interpretable, and impactful tools for healthcare professionals and patients alike.

Chapter 7

References/Bibliography

- [1] World Health Organization. Cardiovascular diseases (CVDs). Available at: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- [2] Framingham Heart Study. About the Framingham Heart Study. Available at: <https://www.framinghamheartstudy.org/about-fhs/>
- [3] Little, R. J. A., & Rubin, D. B. (2002). Statistical analysis with missing data (2nd ed.). John Wiley & Sons.
- [4] Scikit-learn. (n.d.). StandardScaler. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [5] Tukey, J. W. (1977). Exploratory Data Analysis. Addison-Wesley.
- [6] GeeksforGeeks. (2025, June 3). Logistic Regression in Machine Learning. Available at: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>
- [7] Wikipedia. (2025, July 9). Random forest. Available at: https://en.wikipedia.org/wiki/Random_forest
- [8] XGBoost Documentation. (n.d.). Introduction to Boosted Trees. Available at: <https://xgboost.readthedocs.io/en/stable/tutorials/model.html>
- [9] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. Available at: <https://jair.org/index.php/jair/article/download/11192/26406/20731>
- [10] He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.
- [11] IBM. (n.d.). What is a feature engineering? Available at: <https://www.ibm.com/think/topics/feature-engineering>
- [12] Wikipedia. (n.d.). Hyperparameter optimization. Available at: https://en.wikipedia.org/wiki/Hyperparameter_optimization
- [13] Google Developers. (n.d.). Classification: Accuracy, recall, precision, and related metrics. Available at: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>
- [14] Analytics Vidhya. (2019, August 11). 12 Important Model Evaluation Metrics for Machine Learning (2025). Available at: <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>

- [15] GeeksforGeeks. (n.d.). Evaluation Metrics in Machine Learning. Available at: <https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>
- [16] Analytics Vidhya. (2019, August 11). 12 Important Model Evaluation Metrics for Machine Learning (2025). Available at: <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- [17] Precision-Recall Curve. (n.d.). In Wikipedia. Retrieved from https://en.wikipedia.org/wiki/Precision%E2%80%93recall_curve
- [18] Analytics Vidhya. (2020, April 20). How to Deploy Machine Learning Models using Flask (with Code). Available at: <https://www.analyticsvidhya.com/blog/2020/04/how-to-deploy-machine-learning-model-flask/>